

Accuknox Assignment 2: AI/ML Project Experience, ChatBot Architecture and Vector Databases

Zaid Shabir

January 11, 2026

1. Self-Rating and Project Overview

Self-Rating (LLM, DL, AI, ML)

Technology	Rating	Explanation
LLM	C	Limited direct experience; familiar with API usage, prompt design, and AI integration.
Deep Learning (DL)	B	Can code under supervision.
AI	B	Can code under supervision.
ML	A	Can code independently; able to implement end-to-end ML pipelines.

Project Experience

OrchardIntel: Apple Disease Detector with Planet Climate Risk Advisor

- React + TypeScript app for apple leaf disease prediction, dataset management, model training simulation, Planet map viewer, AOI layer visualization, and dynamic farm health scoring.
- Backend managed via Supabase Edge Functions; frontend enhanced with AI-assisted tools: V0 by Vercel, Bolt New by StackBlitz, Copilot.
- AI/ML models provide predictive insights for crop health and climate risk, ensuring accuracy and safety in outputs.

Kraken – Decentralized Messaging Platform

- Secure, decentralized messaging application built on Web3 technologies.
- Users send encrypted messages via Ethereum wallet identities.
- Real-time peer-to-peer communication powered by Gun.js.

AI-Powered Medical Diagnosis System

- Machine learning-based system using SVM, Logistic Regression, and Random Forest to classify medical datasets.
- Handles preprocessing, feature scaling, training/testing splits, and model evaluation (accuracy, precision, recall, F1-score).
- Deployed via Streamlit for interactive real-time predictions.

Other AI/ML experiments, model fine-tuning, and deployment workflows are available on GitHub: <https://github.com/Zai14>.

2. High-Level LLM Chatbot Architecture

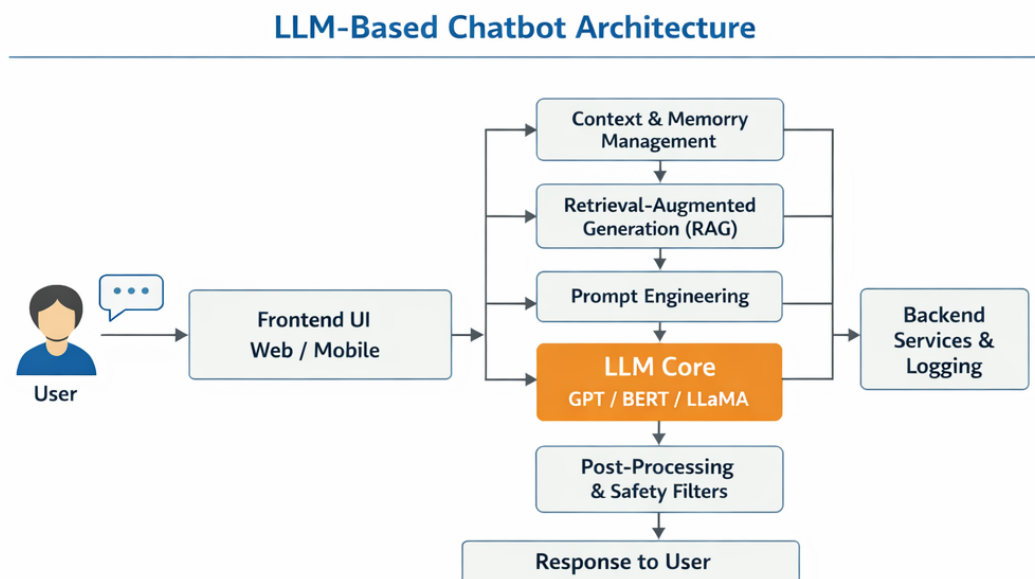


Figure 1: High-Level LLM Chatbot Architecture

Detailed Explanation

The LLM-based chatbot architecture consists of the following components:

- **Frontend UI:** Web or mobile interface; can include AI-assisted prompt workflows using V0, Bolt New, and Copilot.
- **API / Integration Layer:** Manages requests, authentication, session management, and error handling.

- **Context & Memory Management:** Tracks short-term conversation and long-term user preferences for context-aware responses.
- **Prompt Engineering Layer:** Converts user input and context into structured prompts guiding LLM responses; defines tone, persona, and safe response rules.
- **Retrieval-Augmented Generation (RAG):** Optionally fetches external knowledge from a vector database to enhance accuracy.
- **LLM Core:** Processes structured prompts and generates natural language outputs (GPT, BERT, LLaMA, etc.).
- **Post-Processing & Safety Filters:** Ensures output is safe, consistent, and well-formatted.
- **Backend Services & Logging:** Collects analytics, logs interactions, and supports debugging and future improvements.

Data Flow: User input → Frontend → API Layer → Context + RAG → Prompt Engineering → LLM → Post-processing → Backend → Response to User

Advantages: Multi-turn conversation support, domain-specific accuracy via RAG, and reliable, safe responses.

3. Vector Database Overview

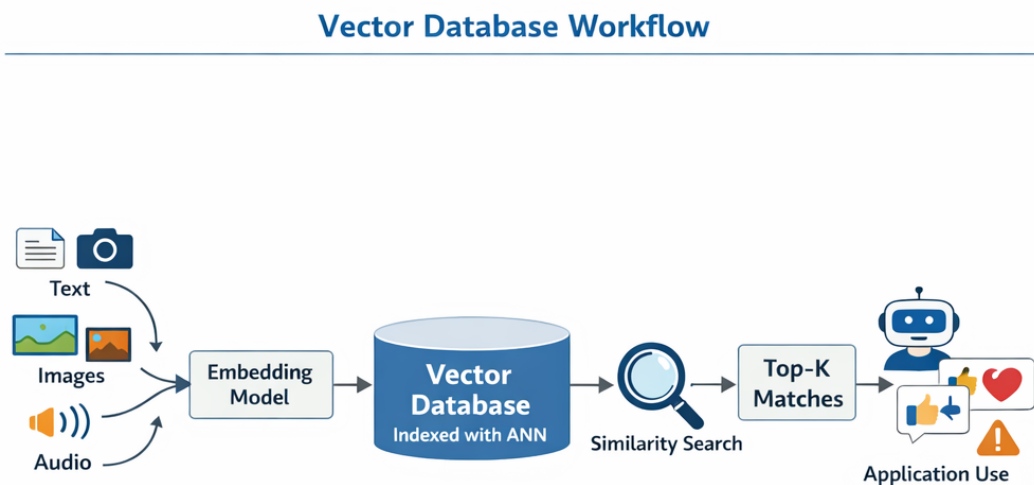


Figure 2: Vector Database Workflow

Detailed Explanation

Vector databases store and manage high-dimensional embeddings representing text, images, or audio.

- **Embedding Generation:** Converts data into vectors using embedding models (Sentence-BERT, CLIP, etc.).
- **Indexing:** Vectors stored in a database with ANN algorithms like HNSW or IVF for fast similarity search.
- **Query Processing:** User queries are embedded and matched to top-K similar vectors based on distance metrics (cosine, Euclidean, dot product).
- **Hybrid Search:** Combines vector similarity with metadata filters (date, type) for precise results.
- **Integration with LLMs:** Retrieved vectors provide additional context to improve LLM-generated responses in chatbots or knowledge bases.

Use Case: Knowledge Base Chatbot

- Build a chatbot for 10,000+ documentation pages and API references.
- Requirements: <100ms response, hybrid search, 1M+ queries/day, cost-effective.
- **Selected Database:** Pinecone (managed, scalable, hybrid search).
- **Alternatives:** Weaviate, Qdrant, Milvus, Chroma, pgvector.

4. Most Complex Python Code Written

AI-Powered Medical Diagnosis System

- **Link:** <https://github.com/zai14/AI-Powered-Medical-Diagnosis-System>
- **Description:** AI-powered Python project implementing ML models, data pre-processing, and predictive analytics. Demonstrates complex Python logic with modular code and classes.

5. Most Complex Database Code Written

Kraken – Decentralized Messaging Database

- **Link:** <https://github.com/zai14/Kraken-Decentralised-Messaging>
- **Description:** Decentralized messaging system with secure storage, authentication, and database integration. Includes complex CRUD operations and data handling.

References

- GitHub: <https://github.com/Zai14>
- Pinecone Documentation: <https://www.pinecone.io/docs/>